# UC32 — Artificial Intelligence Motor Claim Fraud Detector with Photo Evidence

Use Case Specification

| | |
|---|---|
| **Use Case Number** | UC32 |
| **Domain** | Insurance |
| **Category** | General |
| **Primary Module** | Mobile + Application Programming Interface + Admin |
| **Complexity** | High |
| **Estimated Development Hours** | 8 |

## 1. Problem Statement

This use case addresses a problem in the **Insurance** domain. The following summary describes the business pain that the team is expected to solve:

*Motor insurance fraud costs Rs.30,000-40,000 Cr/year. Surveyors inspect only 30%.*

The solution is to be delivered as a mobile + application programming interface + admin. The complexity is rated **High**, so the team should scope their design, implementation, and testing accordingly.

## 2. Users and Roles (Actors)

| Actor | Description |
|---|---|
| **Policyholder** | Submits claims or service requests through the mobile application or the web portal. |
| **Claims Adjuster** | Reviews Artificial Intelligence-generated claim assessments, approves or escalates claims, and updates claim status. |
| **Insurance Administrator** | Configures policy rules, reviews audit logs, and manages payout limits. |
| **System (Automated)** | Performs the Artificial Intelligence processing described in the functional requirements without direct human input. |

## 3. Functional Requirements

### 3.1 Input and onboarding

- The mobile application must allow the primary user to register, authenticate, and provide the input relevant to the Problem Summary above.
- The mobile application must confirm successful submission with a clear on-screen acknowledgement and a reference identifier.

- Input validation must reject malformed or out-of-range data with a clear error message.

### 3.2 Core Artificial Intelligence processing

- The backend must implement the following Artificial Intelligence capabilities, each exposed as a distinct module so that they can be tested independently:
- Damage detection Computer Vision.
- Cost estimation.
- Fraud detection.
- Image forensics.
- When the OpenAI Application Programming Interface is invoked, it is used as follows: GPT-4o Vision analyzes vehicle damage photos (damage type, severity, affected panels); estimates repair costs; compares claim description with photo evidence for inconsistencies; generates surveyor-grade assessment reports.
- Every OpenAI invocation must include a deterministic system prompt version identifier so that responses are reproducible and auditable.

### 3.3 External integrations and data sources

- The solution is self-contained beyond the OpenAI Application Programming Interface; no additional external Application Programming Interfaces are required.
- Pre-trained models and datasets to be used: YOLOv8 for damage detection (train on car damage dataset); GPT-4o Vision as second opinion; image forensics (Error Level Analysis).

### 3.4 Output and user experience

- The mobile application must present the Artificial-Intelligence-generated result in a clearly labelled screen with a plain-language explanation of how the result was derived.
- Every Artificial-Intelligence-generated decision must include a confidence score, and a confidence below a configurable threshold (default 0.6) must trigger an alternative flow (human review, or a fallback non-Artificial-Intelligence answer).

### 3.5 Administrator controls

- The administrator interface must display a real-time summary of volumes, Artificial Intelligence confidence distribution, and error rates.
- The administrator must be able to tune thresholds (for example, confidence thresholds and rate limits) without redeploying the backend.
- The administrator must be able to export the audit log for a selected time range as a Comma-Separated-Values file.

### 3.6 Audit, logging and compliance

- Every Artificial-Intelligence-generated decision must be logged with a timestamp, the input, the output, the model version, and the confidence score.
- Personally identifiable information must be redacted from logs except where retention is legally required.
- The audit log must be tamper-evident (for example, append-only storage or cryptographic checksums).

### 3.7 Performance and deployment

- The end-to-end primary-user flow must complete in under five seconds for a typical input (excluding network latency to external Application Programming Interfaces).
- The backend must be deployable using the standard docker-compose.prod.yml entrypoint shared across the hackathon portal.
- The technology stack used is: React Native, Python (FastAPI), PyTorch, PostgreSQL, OpenAI Application Programming Interface.

## 4. Acceptance Criteria

1. A new user can complete the primary end-to-end flow (from input to Artificial-Intelligence-generated result) in a single session, without requiring any instructions beyond what is visible on the user interface.
2. The module implementing "Damage detection Computer Vision" produces a measurable, testable output on a representative sample input within three seconds.
3. Every Artificial-Intelligence-generated decision is returned with a confidence score, and the user interface displays a plain-language explanation derived from that decision.
4. A confidence score below a configurable threshold (default 0.6) triggers the documented fallback path (human review, alternative model, or a graceful "unable to determine" response) rather than returning an unvetted answer.
5. Every Artificial Intelligence invocation is written to an audit log with timestamp, input, output, model version, and confidence score.
6. The backend tolerates a representative dataset during a fifteen-minute demonstration run without crashes, memory leaks, or exceeding the hardware budget stated in the Technical Notes.
7. All user-facing text expands Artificial Intelligence, Machine Learning, Application Programming Interface and any other domain-specific short forms in their first occurrence on each screen.

## 5. Technical Notes

| | |
|---|---|
| **Artificial Intelligence and Machine Learning components** | Damage detection Computer Vision, cost estimation, fraud detection, image forensics |
| **OpenAI Application Programming Interface usage** | GPT-4o Vision analyzes vehicle damage photos (damage type, severity, affected panels); estimates repair costs; compares claim description with photo evidence for inconsistencies; generates surveyor-grade assessment reports |
| **External Application Programming Interface dependencies** | None (self-contained) |
| **Pre-trained models and datasets** | YOLOv8 for damage detection (train on car damage dataset); GPT-4o Vision as second opinion; image forensics (Error Level Analysis) |
| **Hardware requirements** | Graphics Processing Unit recommended for You Only Look Once object detection; GPT-4o Vision via Application Programming Interface |

| | |
|---|---|
| **Other setup dependencies** | Vehicle damage dataset (Kaggle — ~3000 images); car parts cost database; synthetic claim history |
| **Technology stack** | React Native, Python (FastAPI), PyTorch, PostgreSQL, OpenAI Application Programming Interface |
| **Estimated development hours** | 8 |

## 6. Glossary

Short-form technical names referenced above are described here so that the team can work without needing to look them up.

| | |
|---|---|
| **GPT-4o** | OpenAI's multimodal (text, image, audio) version of Generative Pre-trained Transformer 4. |
| **YOLOv8** | You Only Look Once version 8 — a real-time object-detection model family. |